음성 AI 전문가 과정 · Day 30 / 40

Day 30

LiveKit 심화 & Speech-to-Speech
Handoff · SIP 전화 · Realtime · Gemini Live

```python
class Hana(Agent):
    @function_tool
    async def transfer_to_billing(self, context):
        return Billing(chat_ctx=self.chat_ctx), "…"

@server.rtc_session(agent_name="hanbit-phone")
async def phone_entrypoint(ctx): ...

AgentSession(llm=RealtimeModel())   # S2S
```

도구 호출 → 예약 · 수납 담당 Handoff → 전화 음질 · SIP 요청서 → S2S 모델 꽂기 → 전화 상담 리허설
6 주차 M6 마지막 날 · 4 시간 · livekit-agents 1.8.10

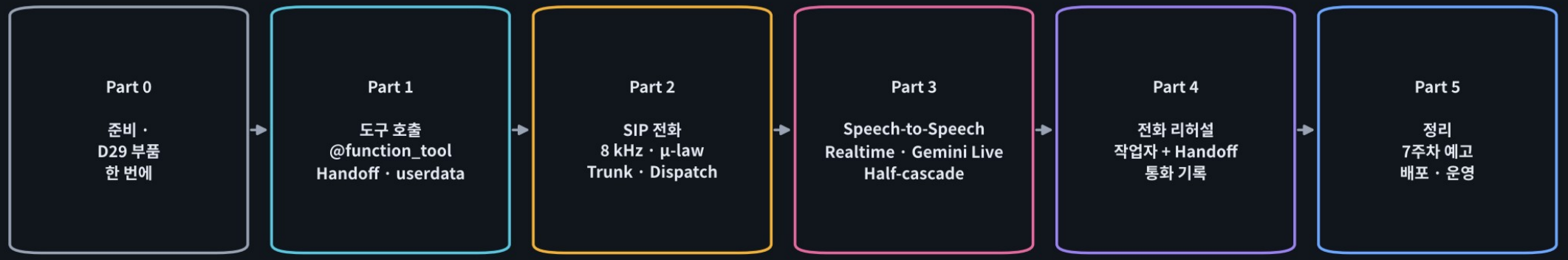
0.0 · 오늘 답할 질문

비서를 병원 대표번호로 개통하려면

1	정확한 가격 · 예약 저장은 누가 하나 ?	Part 1
2	예약 담당에서 수납 담당으로 어떻게 넘기나 ? 정보는 ?	Part 1
3	전화는 어떤 길로 방에 들어오나 ?	Part 2
4	전화 음질은 받아쓰기에 어떤 영향을 주나 ?	Part 2
5	STT · LLM · TTS 를 한 모델로 바꾸면 무엇이 달라지나 ?	Part 3
6	전화 한 통을 처음부터 끝까지 리허설하려면 ?	Part 4

담당 나누기 → 전화 → 머리 바꾸기 → 리허설

D30 · 담당을 나누고(Handoff) · 전화로 받고(SIP) · 머리를 바꿔 보고(S2S) · 한 통화로 리허설



같은 entrypoint · 같은 대본에 '담당 여러 명 + 도구', '전화 참가자', 'S2S 머리'를 더한다

교시	내용	빈칸	애니메이션
1 교시	Part 0 ~ 1 도구 호출 · @function_tool · userdata · Handoff	1 ~ 4	D30-1 · 2 · 3
2 교시	Part 2 SIP 구조 · 전화 음질 · 요청서	5 · 6 · 7	D30-4 · 6
3 교시	Part 3 S2S · Half-cascade · 능력 표	8 · 9	D30-7 · 8
4 교시	Part 4 ~ 5 전화 리허설 · 통화 기록 · 정리	–	D30-5 · 9

LLM 은 주문서만 쓰고 , 일은 우리 함수가 한다

1. 세션 → LLM: 대화 + 도구 설명서
2. LLM → FunctionToolCall(name, arguments)
3. 세션이 우리 함수 실행 → 결과를 대화에 붙임
4. 세션 → LLM 다시 : 결과를 보고 말 만들기

Qwen3 글자	LiveKit 조각
네 , 예약해 드릴게요 .	`ChoiceDelta(content=…)`
`<tool_call>{…}</tool_call>`	`ChoiceDelta(tool_calls=[FunctionToolCall])`

🧩 **빈칸 1**

lk_llm.FunctionToolCall(name=..., arguments=JSON 글자 , call_id=...)

⚖️ **로컬 Qwen: 도구가 있으면 글자를 끝까지 모아 판단 → 첫 글자 (TTFT) 가 늘어나는 대가**

▶ 애니메이션 D30-2 도구 호출 한 바퀴

도구가 Agent 를 돌려주면 담당이 바뀐다



🍎 병원 창구

접수처가 " 비용은 수납 창구에서요 " 하며 환자 차트 (userdata) 를 들려 보낸다 .

▶ 애니메이션 D30-3 Handoff와 공유 메모

함수 이름 · docstring · 타입 힌트 = LLM 용 설명서

```python
class Hana(Agent):
    @function_tool
    async def book_appointment(self, context: RunContext[CallInfo],
                               date: str, time: str, service: str, name: str) -> str:
        '''예약을 확정합니다. 날짜 · 시간 · 진료 · 이름이 모두 확인된 뒤에만.'''
        info = context.userdata
        info.bookings.append({...}); return "예약 완료. 예약 번호 HB-0001 …"

    @function_tool
    async def transfer_to_billing(self, context):
        return Billing(chat_ctx=self.chat_ctx), "수납 담당이 안내해 드릴게요."
```

🧩 빈칸 2·3

function_tool (도구로 만들기)

userdata (공유 메모)

담당	도구
Hana	book_appointment transfer_to_billing
Billing	get_price transfer_to_booking

사건 네 종류로 통화 흐름을 읽는다

사건	내용 (예)
FunctionCall	book_appointment({"date": " 다음 주 수요일 ", ...})
FunctionCallOutput	예약 완료 . 예약 번호 HB-0001
ChatMessage	네 , 예약해 드렸습니다 …
FunctionCall	transfer_to_billing({})
AgentHandoff	Hana → Billing
FunctionCall	get_price({"service": " 스케일링 "})
ChatMessage	보험 적용 시 약 1 만 5 천 원 …

```
results[0].expect.contains_function_call(
    name="book_appointment")
results[1].expect.contains_agent_handoff(
    new_agent_type=Billing)
```

🧩 **빈칸 4**

contains_agent_handoff

❓ **대화를 안 넘기면 ? 다 넘기면 ? → 최근 몇 턴 + 요약 + userdata**

대화 (chat_ctx) · 공유 메모 (userdata) · 목소리

넘기는 방식	수납 담당의 첫마디	입력 토큰	쓰는 때
chat_ctx=self.chat_ctx (전체)	" 스케일링 비용 말씀이시죠 ? ..."	많음	짧은 통화
안 넘김 (userdata 만)	" 무엇을 도와드릴까요 ?" (다시 묻기)	적음	거의 쓰지 않음
최근 몇 턴 + 요약	" 스케일링 비용 말씀이시죠 ? ..."	적당	긴 통화 · 실무

1. 담당마다 목소리를 다르게 : Agent(tts=KoTTS(VOICE_BILLING)) → 전화로도 담당이 바뀐 것을 안다
2. 최소 권한 : Billing 에는 예약 도구가 없고 , 필요하면 transfer_to_booking

전화 건 사람도 방의 참가자 한 명 (kind = SIP)

전화가 방에 들어오는 길: PSTN → SIP Trunk → LiveKit SIP → Dispatch Rule → Room ← Agent



인바운드(걸려 온 전화): 왼쪽 → 오른쪽

아웃바운드(비서가 거는 전화): create_dispatch(방에 비서) → CreateSIPParticipant(sip_call_to=환자 번호) → Outbound Trunk → 환자 휴대폰

소리는 8 kHz · G.711 μ-law (300 ~ 3,400 Hz) → 비서의 귀(STT)는 '전화 음질'을 듣는다

🍎 국제 전화 교환원

SIP Trunk 가 전화망 말을 인터넷 말로 통역해 방 문 앞까지 , Dispatch Rule 이 " 몇 호실 · 상담원 하나 ".

▶ 애니메이션 D30-4 전화가 들어오는 길

8 kHz · μ-law 8 비트 : 300 ~ 3,400 Hz 만 남는다

```
PHONE_SR = 8000
MU = 255

def mulaw_8bit(x, mu=MU):
    y = np.sign(x) * np.log1p(mu*np.abs(x)) / np.log1p(mu)
    q = np.round((y + 1) / 2 * 255) / 255 * 2 - 1
    return np.sign(q) * ((1 + mu) ** np.abs(q) - 1) / mu
```

🧩 **빈칸 5 · 6**

8000 (전화망 샘플링) · 255 (μ)

작은 목소리 8 비트	SNR
그냥 자르기	약 14 dB
μ-law	약 37 dB

📐 $y = \text{sign}(x) \cdot \ln(1 + \mu|x|) / \ln(1 + \mu)$ → 작은 소리는 크게 펴서 8 비트로도 덜 뭉개진다

▶ 애니메이션 D30-6 전화 음질 만들기

Inbound Trunk · Dispatch Rule · CreateSIPParticipant

```
inbound = api.CreateSIPInboundTrunkRequest(trunk=api.SIPInboundTrunkInfo(
    name="hanbit-inbound", numbers=[CLINIC_NUMBER], krisp_enabled=True))
dispatch = api.CreateSIPDispatchRuleRequest(name="hanbit-dispatch",
    rule=api.SIPDispatchRule(dispatch_rule_individual=
        api.SIPDispatchRuleIndividual(room_prefix="call-")),
    room_config=api.RoomConfiguration(agents=[
        api.RoomAgentDispatch(agent_name="hanbit-phone")]))
outbound = api.CreateSIPParticipantRequest(sip_trunk_id="ST_…",
    sip_call_to=PATIENT_NUMBER, room_name="call-out-0001",
    participant_identity="patient-minjun", wait_until_answered=True,
    hide_phone_number=True)
```

🧩 빈칸 7

sip_call_to (걸 상대 번호)

규칙	방
Individual	전화마다 새 방
Direct	모두 한 방
Callee	받은 번호별

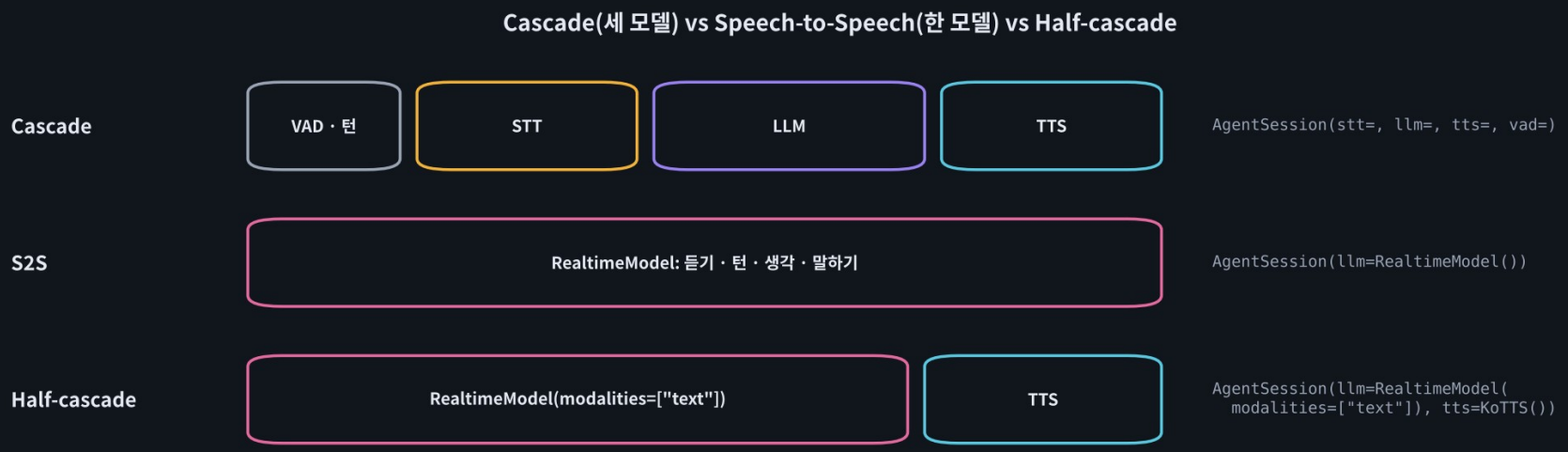
agent_name 으로 명시적 배정 · SIP 참가자 속성

```python
@server.rtc_session(agent_name="hanbit-phone")
async def phone_entrypoint(ctx: JobContext):
    await ctx.connect()
    caller = await ctx.wait_for_participant()
    if caller.kind == rtc.ParticipantKind.PARTICIPANT_KIND_SIP:
        phone = caller.attributes.get("sip.phoneNumber")
    await session.start(agent=Hana(), room=ctx.room)
    session.say("…AI 상담원이 응대하며 상담 내용이 기록됩니다… ")
```

SIP 속성	뜻
sip.phoneNumber	발신 번호
sip.callStatus	ringing → active → hangup
sip.trunkPhoneNumber	받은 번호

1. DTMF(숫자 버튼): sip_dtmf_received 이벤트로 메뉴
2. 사람에게 넘기기 : SIP 전화 돌려주기 (transfer) 도구

Cascade = 통역사 세 명 , S2S = 원어민 한 명



	Cascade	S2S
지연	단계마다 쌓임	짧음
제어	글자 검사 · 정규화 쉬움	어려움
목소리 · 로컬	아무 TTS · 로컬 가능	제공 목소리 · 클라우드 전용

같은 대본 · 도구 , llm 자리만 바꾼다

```
rt_openai = lk_openai.realtime.RealtimeModel(voice="marin")
rt_gemini = lk_google.realtime.RealtimeModel(voice="Kore")
rt_half   = lk_openai.realtime.RealtimeModel(modalities=["text"])

AgentSession(llm=rt_openai)                 # S2S
AgentSession(llm=rt_half, tts=KoTTS())      # Half-cascade
```

🧩 **빈칸 8 · 9**

RealtimeModel · modalities

supports_say=False → 인사는 generate_reply(instructions=...)

▶ 애니메이션 D30-7 Cascade vs S2S

능력 값이 세션 동작을 바꾼다

능력	OpenAI	Gemini	Half	뜻
audio_output	True	True	False	False 면 세션이 우리 tts= 로 말함
turn_detection	True	True	True	모델이 스스로 턴 판단 (VAD 불필요)
auto_tool_reply_generation	False	True	False	도구 결과 뒤 모델이 알아서 이어 말함
message_truncation	True	False	True	끼어든 곳까지만 기록
supports_say	False	False	False	session.say 불가

▶ 애니메이션 D30-8 능력 표 읽기

Cascade · Half-cascade · S2S

무엇을 고를까: 빠름 · 자연스러움(S2S) vs 제어 · 비용 · 로컬(Cascade)

Cascade

· 숫자 · 고유명사 읽기 제어
· 아무 목소리(복제 포함)
· 로컬 · 온프레미스 가능
· 단계별 교체 · 디버깅 쉬움

Half-cascade

· 빠른 귀 · 턴 판단 · 생각
· 목소리 · 숫자 읽기는 우리 TTS
· S2S보다 조금 느림
· 클라우드 필요

Speech-to-Speech

· 가장 짧은 지연
· 말투 · 감정 반영
· 숫자 · 발음 제어 어려움
· 오디오 토큰 비용 · 클라우드 전용

❓ 한빛치과 전화 비서 : 예약 번호 · 가격을 틀리면 안 되고 , 정보가 외부로 나가면 안 된다면 ? → Cascade(로컬) 또는 Half-cascade

전화 음질 + Handoff + 작업자 + 통화 기록

```python
@phone_server.rtc_session(agent_name=AGENT_NAME)
async def phone_entrypoint(ctx: JobContext):
    session = AgentSession(stt=WhisperSTT(), llm=make_llm(), tts=KoTTS(),
        vad=VAD, userdata=CallInfo(),
        turn_handling={"endpointing": {"min_delay": 0.8}})
    if ctx.is_fake_job(): attach_fake_io(session, PHONE_VOICES, t0)
    ctx.add_shutdown_callback(on_end)       # 기록 저장
    await session.start(agent=Hana(), room=ctx.room)
    session.say("…AI 상담원이 응대하며 … ")

await phone_server.simulate_job("call-rehearsal", fake_job=True)
```

결과 (예)	
담당	Hana → Billing
예약	HB-0001
e2e 중앙값	약 2.6 초 *

* 테스트 대역 기준 . Colab T4 에서 직접 확인

▶ 애니메이션 D30-5 전화 한 통의 일생

history · userdata · usage 를 call_....json 으로

종류	내용 (예)
assistant	한빛치과입니다. 이 통화는 AI 상담원이 …
도구 주문	book_appointment({"date": ...})
도구 결과	예약 완료. 예약 번호 HB-0001
담당 교체	hana → billing
도구 주문	get_price({"service": " 스케일링 "})
assistant	스케일링은 보험 적용 시 약 1만 5천 원 …

1. 최소 수집
2. 전화번호 마스킹 · hide_phone_number
3. 보관 기간 · 접근 권한
4. 고지 문장도 기록에 남김

create_dispatch → create_sip_participant

```
await lkapi.agent_dispatch.create_dispatch(
    api.CreateAgentDispatchRequest(agent_name=AGENT_NAME, room=room))
await lkapi.sip.create_sip_participant(api.CreateSIPParticipantRequest(
    sip_trunk_id=TRUNK, sip_call_to=DEMO_PHONE, room_name=room,
    wait_until_answered=True, hide_phone_number=True))
```

⚠ 강사 계정 · 강사 휴대폰만 . 번호는 출력하지 않음 (" 있음 " 만)

방식	말 끝 → 첫 소리	도구 · Handoff	숫자 읽기	목소리	로컬
Cascade	오늘 측정	✔	✔ normalize_ko	✔ 아무 TTS	✔
S2S	키가 있으면 측정	✔ 같은 Agent	✘ 모델에 맡김	△ 제공 목소리	✘

정리 · 6 주차 마무리

Handoff · SIP 참가자 · RealtimeModel

개념	코드
도구 주문서	`lk_llm.FunctionToolCall`
도구	`@function_tool`
공유 메모	`context.userdata`
Handoff 시험	`expect.contains_agent_handoff`
전화 음질	`PHONE_SR = 8000 · MU = 255`
전화 걸기	`CreateSIPParticipant(sip_call_to=…)`
S2S	`RealtimeModel(modalities=…)`

다음 : D31 FastAPI 음성 API 서버

STT · TTS REST · WebSocket 엔드포인트와 비동기 처리 . 6 주차의 실시간 비서를 서비스로 띄우는 첫걸음입니다 .

`worksheet.html` 40 문항

개념 워크시트 · 노션 제출

`animation.html` 9 개 탭

인터랙티브 애니메이션

`D30_LiveKit 심화 _S2S_ 실습 .ipynb` 빈칸 9

실습 노트북 · 키 권장

온라인 자료 imguru-mooc.github.io/AI_Voice/day30